# Informatics II, Spring 2026, Solution 1

## Task 1

```
#include <stdio.h>
int main(){
    int a[]={15,5,1,4,2,8,7},n=7,t;
    for(int i=0;i<n-1;i++){
        for(int j=0;j<n-1-i;j++)
            if(a[j]>a[j+1]){t=a[j];a[j]=a[j+1];a[j+1]=t;}
        for(int k=0;k<n;k++)printf("%d ",a[k]);
        printf("\n");
    }
}
```

## Task 2

$A = (4, 8, 8, 7, 7, 3, 5, 0, 5)$

- start of the outermost loop

$A = (0, 4, 8, 8, 7, 7, 3, 5, 5)$

$A = (0, 4, 8, 7, 7, 3, 5, 5, 8)$

- 1st completion of the outermost loop

$A = (0, 3, 4, 8, 7, 7, 5, 5, 8)$

$A = (0, 3, 4, 7, 7, 5, 5, 8, 8)$

- 2nd completion of the outermost loop

## Task 3

Given order $i$ of Hilbert curve on $l \times l$ canvas, the length of the curve $H(i)$ can be recursively computed by the formula:

$$\begin{cases} H(i) = 2 * H(i-1) + 3 * l/2^i \\ H(0) = 0 \end{cases} \tag{1}$$

They are obtained as follows: $H(i) = \frac{1}{2} * 4 * H(i-1) + 3 * l/2^i$

A Hilbert curve at level $i$ is made up of four curves of order $i-1$, each scaled down by a factor of $1/2$, plus three additional connecting lines.

So,

$$
\begin{aligned}
H(15) &= 2 * H(14) + 3 * l/2^{15} \\
&= 2 * (2 * H(13) + 3 * l/2^{14}) + 3 * l/2^{15} \\
&= 2^{15} * H(0) + 3 * l/2^{15} + 2 * 3 * l/2^{14} + ... + 2^{14} * 3 * l/2^{1} \\
&= 3l * (2^{-15} + 2^{-13} + 2^{-11} + ... + 2^{13}) \\
&= -l * 2^{-15}(1 - 2^{30}) \\
&= l(2^{15} - 2^{-15}) \\
&\approx 163839\,\text{cm}
\end{aligned}
$$

or you can implement the recursion with code to get the value.

# Task 4

See code in the file *matrix.c*

# Task 5

### 5.1 & 5.2

```
1
2  //-----------------------------------------------------------
3
4  bool pairSum() {
5    for (i = 0; i < n; i++) {
6      for (j = i+1; j < n; j++) {
7        if (A[i] + A[j] == c) { return true; }
8      }
9    }
10   return false;
11 }
12
13 //-----------------------------------------------------------
14
15 bool pairSumSorted() {
16   i = 0; j = n - 1;
17   while (i < j) {
18     if (A[i] + A[j] == c) { return true; }
19     else if (A[i] + A[j] < c) { i++; }
20     else { j--; }
21   }
22   return false;
23 }
```

**5.3** The worst case runtime of algorithms `pairSum` is $O(n^2)$ and `pairSumSorted` is $O(n)$ for an array with n elements. See code in the file *pairSum.c*.